

Dev Log — 23 July 2026

Session Goal

Make `Action` / `KeyMap` dynamically sized and definable per-game instead of the fixed `enum class Action` + array-shaped `KeyMap` from the previous session — the goal stated at the top of this session was explicitly "how do I change the shape of `KeyMap` and `Actions` so they can be dynamically sized and created on a per game basis." This ran as a long design conversation (captured in `design-docs/2026-07-23_125432_dynamic-keymap-action.md`, revised repeatedly over the session as the shape changed) in parallel with the user hand-writing the actual implementation per `CLAUDE.md`'s read-only-for-the-agent policy. Landed in one commit (`a15734d`, "wip: upgrading input actions and keybindings") — the message's own "wip" is accurate: several pieces documented as decided in the design doc are not yet reflected in the committed code, and the commit does not currently compile. This log distinguishes what the diff actually contains from what the design doc describes as the target, since they diverged by the end of the session.

Also fixed this session, unrelated to input: both `architecture-report` and `code-review-report` skills no longer auto-render a PDF (or a PNG diagram preview) — they write the `.md` and stop, so the doc can be reviewed before `/report-pdf` is run manually. `dev-log` (this command) was checked and already had no such step.

Work Completed

Core shape change: `Action`, `KeyMap`, `ActionSet` — landed and compiles

`Action` (`core/input/action.hpp`) is now a plain `struct{ int id; string name; ActionValueType type; bool isComposite; }` — no enum, no fixed-size `keycodes` array. `KeyMap` (`core/input/keymap.hpp`) is `unordered_map<int, vector<KeyBinding>>`, keyed by `id` rather than by `Action` itself, so no `operator==` / `std::hash` is needed on `Action` anywhere in the input layer. A new `ActionSet` (`core/input/action-set.{hpp,cpp}`) provides `RegisterAction(name, type)` (auto-incrementing `id`), `Get(name)` (asserts on an unregistered name rather than returning a default `Action`), `All()`, and `BuildKeyMap(vector<ActionBindingBuilder>)`. `ActionSet` lives on `Engine::Application` (not `SnakeApplication`) so every game gets one for free; `IApplication::GetInputActions()` is `const`-only on the interface (`i-application.hpp:47`), with a second, non-virtual, non-const overload added directly on `Application` (`application.hpp:84-85`) for `Configure()` to register into — standard const/non-const overloading, not a const/mutable split on the field itself (`inputActions` stays `private`).

`InputDispatcher`, `InputSystem`, and `KeyboardInputSource` were all updated to match: `InputDispatcher`'s constructor takes `vector<Action>` `actions` and sizes two `vector<bool>` (`lastFrameDown` / `currentDown`) from it instead of the old `array<bool, Action::Count>`; `Run()` / `IsActionDown()` iterate/scan that vector instead of casting an index to the old enum; `KeyboardInputSource::IsActionDown` now does `keyMap.find(action.id)` (not `operator[]`, since the

method is `const`). `Application::Initialize()` passes `config.engine.input.actions` as a fourth constructor argument (`application.cpp:80-82`). This chain compiles and is consistent end to end.

Two bugs the previous session's log had flagged as open (UE-0036) are fixed as part of this rewrite, confirmed directly in `input-dispatcher.cpp : Run()` now publishes `InputActionPressedEvent{action}` / `InputActionReleasedEvent{action}` with the real transitioning action (previously default-constructed, always resolving to enumerator `0`), and `this->lastFrameDown[i] = isDown` is now actually assigned at the end of the loop (previously never written at all, so every frame looked like a fresh transition). Flagging both for confirmation rather than closing outright, per this log's own ticket-lifecycle rule — see Known Issues below.

ActionBindingBuilder — chainable binding authoring, landed

`core/input/action-binding-builder.{hpp,cpp}` implements the `Bind(name).Axis1D(keys, scale).Axis1D(keys, scale) / .Digital(keys)` chainable builder discussed at length this session, replacing raw positional/aggregate-init `KeyMap` construction. `ActionSet::BuildKeyMap` consumes a `vector<ActionBindingBuilder>` and validates each binding's contribution type against the action's declared `ActionValueType` via an `assert` (`action-set.cpp:39-44`) before it's a runtime surprise — except see UE-0039 below, which breaks that validation for exactly the case it exists to catch. `SnakeApplication::Configure()` was rewritten to use it: `MoveForward / MoveRight` as `Axis1D` actions (opposite-signed key groups — `W / Up : +1`, `S / Down : -1`, etc.), `Confirm / Decline` as `Boolean`.

ActionValue / ActionValueType — landed with a real bug (UE-0039)

`ActionValue` is `std::variant<bool, float, Vector2D>` (`core/input/action-value.hpp`), matching the session's decision to use a variant rather than Unreal's uniform tagged-vector approach (this codebase has no generic modifier/trigger pipeline that needs uniform storage). The `ValueTypeOf(ActionValue) -> ActionValueType` helper — needed by `BuildKeyMap`'s validation, and originally sketched as pseudocode mid-session before being pointed out as such — has been implemented, but with a copy-paste bug: its `if constexpr` chain checks `std::is_same_v<T, bool>` twice (once correctly for the `Boolean` branch, then again — instead of `float` — for what should be the `Axis1D` branch). See UE-0039.

ActionBindings — declared, not implemented, not wired to anything (UE-0040/0041)

`core/input/action-bindings.hpp` exists with the `Bind / BindAxis / Update` shape discussed this session, but diverges from where the design doc landed and isn't functional yet:

- No `action-bindings.cpp` — every method is declared, none defined. Nothing that includes this header and calls a method would link.
- `Bind / BindAxis` return `void`, not the move-only RAIL `Handle` the session's design settled on — the two problems that motivated `Handle` (two listeners on one action clobbering each other via `unordered_map<int, function>`; a destroyed `Snake` leaving a dangling callback behind after `SceneManager::SwitchTo`'s `Transient`-scene teardown, `scene-manager.cpp:23-28`) are both still live risks in this shape.

- `BindAxis(name, std::function<void()> callback)` doesn't even accept the axis value — the callback signature has no way to receive what changed.
- It's never constructed anywhere. `InputSystem` doesn't hold one; nothing in `application.cpp` builds one. `Snake` doesn't reference it at all — instead `Snake` was wired directly to `IInputSystem / ActionSet` (see below), which is exactly the ownership the design doc argues against in its own §8 ("Snake shouldn't hold ActionSet/IInputSystem directly... that's the wrong owner for that job"), written in the same commit.

Snake movement — event subscription removed, replacement doesn't compile (UE-0042/0043/0044)

The old `Subscribe<InputActionPressedEvent> / switch(e.action) / SetDirection(...)` block in `Snake`'s constructor was deleted entirely (`snake.cpp`). In its place, `Snake::Move()` now does:

```
float vertical = std::get<float>(this->inputSystem->GetActionValue(this->inputActions.Get("MoveVertical")));
float horizontal = std::get<float>(this->inputSystem->GetActionValue(this->inputActions.Get("MoveHorizontal")));
```

Three separate problems here, each independently blocking:

- `inputSystem` is a reference member (`const Core::Input::IInputSystem& inputSystem` , `snake.hpp`), not a pointer — `->` isn't valid on a reference to a type with no overloaded `operator->` ; needs `..`.
- `IInputSystem::GetActionValue` doesn't exist. `i-input-system.hpp` wasn't touched this session — it still declares nothing beyond a virtual destructor (confirmed by reading the file directly, not just the diff). This doesn't compile regardless of the `-> / ..` fix.
- Even once both of those are fixed, the action names don't match: `Get("MoveVertical") / Get("MoveHorizontal")` are looked up, but `SnakeApplication::Configure()` only ever registers `"MoveForward" / "MoveRight"` . `ActionSet::Get` asserts on an unregistered name (`action-set.cpp:26`) — this would abort at runtime in a debug build, or return past-the-end/undefined behaviour in a release build with assertions compiled out.

And separately from all three: even if `vertical / horizontal` resolved to real values, `Move()` never does anything with them — no assignment to `this->direction` , no call to `SetDirection` . Between the deleted subscription and this, nothing currently updates `Snake`'s direction from input at all. Movement is non-functional in this commit independent of the compile errors.

Build: action-binding-builder.cpp missing from CMakeLists.txt

`action-set.cpp` was added to the executable's source list (`CMakeLists.txt:47`), but `action-binding-builder.cpp` — which `action-set.cpp` depends on via `ActionBindingBuilder` — was not. Every `ActionBindingBuilder` method (`Digital / Axis1D / Axis2D / constructor`) would be an undefined reference at link time.

Lessons Learned

`if constexpr` Type-Dispatch Chains Don't Warn on a Duplicated Branch Condition

`ValueTypeOf`'s bug (checking `is_same_v<T, bool>` twice instead of `bool` then `float`) compiles cleanly and produces no warning — the compiler has no way to know the second branch was supposed to test a different type; from its point of view, an unreachable-but-well-formed `else if` is not an error. Worth remembering as a general pattern risk with hand-written `if constexpr` dispatch over several types: nothing catches "this branch's condition is identical to an earlier one" short of actually exercising every alternative in a test.

`.` vs `->` on a Reference Member Is Silent Until It Isn't

`this->inputSystem->GetActionValue(...)` against a `const IInputSystem&` member is exactly the kind of typo that's easy to make when copying a pointer-shaped sketch (several versions of this session's design discussion used `IInputSystem*` /pointer language interchangeably with reference language before settling) — the fix is mechanical once spotted, but nothing about the surrounding code signals which one is correct without checking the member's actual declared type.

Confirming Behaviour by Reading the Code Beats Assuming It

Mid-session, a question came up about whether `ActionBindings::Bind / BindAxis` calls happening after `Initialize()` (rather than during `Configure()`) was safe, given a future `MainMenu` scene might delay `Snake`'s construction significantly. Rather than reasoning about it abstractly, this was checked directly against `SceneManager::SwitchTo / ResolveScene` (`scene-manager.cpp:17-36`, `102-116`), confirming scene construction is synchronous with no pending/deferred-switch mechanism anywhere in `SceneManager`. That direct read also surfaced the actual, separate hazard worth caring about — `Transient` scene teardown (`entry.instance.reset()`) destroying `Snake` while it may still be registered in `ActionBindings` — which motivated the `Handle` /RAII design in the doc, not yet in the code (see UE-0041).

Architectural Decisions

`KeyMap` Keys on `int id`, Not `Action` Itself

Considered giving `Action` `operator==` / `std::hash` so `KeyMap` could be `unordered_map<Action, ...>` directly. Rejected — `Action` already carries a stable `id` for exactly this purpose, and keying on the plain `int` avoids adding identity/hashing machinery to a type that's otherwise just a value-carrying struct. The `int` is also what stays cheap to compare in `InputDispatcher::Run()`'s per-frame hot path.

ActionSet Lives on Application, Not SnakeApplication

`ActionSet::Get(name)` is only useful if the instance it was registered on outlives `Configure()` — a game entity constructed later (e.g. `Snake`) needs something to still look names up on. Landed on making `ActionSet` a member of the engine-level `Application` base rather than any one game's subclass, so every game gets the mechanism without re-deriving it, with a `const`-only accessor on the public `IApplication` interface (read-only for anything holding the interface) and a second non-virtual non-const overload on the concrete `Application` class for subclasses' `Configure()` to register into.

ActionBindings Should Own Bind / BindAxis via a Move-Only Handle, Not void

Decided (documented in the design doc) but not yet applied in the committed `action-bindings.hpp`: registration should return an RAI `Handle` that unregisters on destruction, and storage should move from "one-callback-per-action-id" to a flat list keyed by a unique per-registration id. Two problems drove this: multiple listeners on the same action clobbering each other under the current `unordered_map<int, function>` shape, and a destroyed `Snake` leaving a dangling callback behind once any scene other than `Gameplay` exists and `Transient` teardown runs. Next session's implementation of `ActionBindings` should use this shape rather than the simpler one currently declared.

ActionValue as std::variant, Not Unreal's Uniform Tagged Vector

Confirmed this session, applied in the actual code (`action-value.hpp:11`). Unreal always stores an action's value as a 3-float vector plus a type tag, because its modifier/trigger pipeline needs to process any action's value generically as raw float components. This codebase has no equivalent generic pipeline — each consumer expects one specific shape — so `std::variant`'s compile-time-checked `std::get<T>` access was chosen over a type that can be silently misread at runtime.

Known Issues Carried Forward

- **UE-0014** — REAPER: `GameState` still owns stale snake/apple references and polls input. Unchanged.
- **UE-0015** — REAPER: `ManagerInterface` update/debug methods commented out. Unchanged.
- **UE-0016** — REAPER: `SnakeGameSettings::GetBoxSize()` commented out. Unchanged.
- **UE-0019** — SIDE QUEST: `BoundaryWrapSystem`. Unchanged.
- **UE-0020 / UE-0023** — REAPER: Remove stale `stateMachine` `shared_ptr` from `Application` members. Not investigated this session.
- **UE-0021** — 1UP: `std::is_abstract_v` discriminator breaks for `ColliderComponent2D`. Not investigated.

- **UE-0024** — REAPER: UI component system commented out. Not investigated.
- **UE-0026** — REAPER: Remove `Apple` verification hooks before merge. Not investigated.
- **UE-0028** — MAIN QUEST: `GameStateMachine` use-after-move. Unchanged, still entangled with UE-0035.
- **UE-0030** — 1UP: `TransformComponent2D::GetTransform()` returns a shared function-local `static`. Not touched.
- **UE-0031** — REAPER: Dependency injector correctness bugs. Not touched.
- **UE-0032** — REAPER: `ComponentDispatcher` `const_cast` ing. Not touched.
- **UE-0033** — REAPER: Miscellaneous cleanups from code review. Not touched.
- **UE-0029** — RAID BOSS: Unify entity/component ownership to remove the leak and double free. Unchanged, not touched by this session's input work.
- **UE-0034** — REAPER: `Snake::Update` is never called from anywhere Still true — `Snake::Update` (`snake.cpp:61`) has zero callers. More relevant now than last session: `Snake::Move()` (called from `Update`) contains this session's new input-reading code (see UE-0042/0043), which is therefore also currently unreachable regardless of whether it compiles.
- **UE-0036** — Event's `action` field never populated Flagging for confirmation rather than closing outright, per this log's usual rule. Diff shows both halves of this fixed: `InputDispatcher::Run()` now publishes `InputActionPressedEvent{action}` / `InputActionReleasedEvent{action}` with the real action (`input-dispatcher.cpp` , replacing bare default-constructed events), and `lastFrameDown[i] = isDown` is now assigned at the end of the loop (previously never written). Recommend closing once confirmed — the boolean/digital half of the input pipeline (not the broken axis-reading half in `Snake::Move`) looks correct end to end.
- **UE-0037** — Keymap array literal order didn't match `Action` enum order Flagging for confirmation rather than closing. The underlying mechanism this bug depended on — a positional `array` / `vector` keyed to enum ordinal — no longer exists; `KeyMap` is now keyed by `id` via named `ActionSet::RegisterAction` / `BuildKeyMap` calls, which can't have this failure mode structurally. Recommend closing as superseded by the redesign rather than "fixed" per se.

Next Session

- **UE-0039** — BUG: `ValueTypeOf` checks `bool` twice instead of `bool` then `float` `action-value.hpp` 's `ValueTypeOf` has two `if constexpr (std::is_same_v<T, bool>)` branches — the second should test `float` (for `ActionValueType::Axis1D`) but doesn't, so it's unreachable and every `float` -valued `ActionValue` falls through to the final `return ActionValueType::Boolean;` . This directly breaks `ActionSet::BuildKeyMap` 's validation `assert ( action-set.cpp:39-44 )` for the exact case it exists to catch: `MoveForward` / `MoveRight` are registered as `Axis1D` but bound via

`.Axis1D(...)` (`float` `contributions`, `action-binding-builder.cpp`), so `ValueTypeOf(contribution)` reports `Boolean` while `action.type` is `Axis1D` — the assert fails on every `Configure()` call once this path is exercised. High priority; blocks `BuildKeyMap` for any axis action.

- **UE-0040** — BUG: `action-binding-builder.cpp` missing from `CMakeLists.txt` `action-set.cpp` was added to the build (`CMakeLists.txt:47`) but `action-binding-builder.cpp` — which it depends on for `ActionBindingBuilder` 's constructor/ `Digital` / `Axis1D` / `Axis2D` — was not. Link failure the moment anything using `ActionBindingBuilder` is actually built. Add the missing source file.
- **UE-0041** — MAIN QUEST: `ActionBindings` has no implementation and isn't wired to anything `core/input/action-bindings.hpp` declares `Bind` / `BindAxis` / `Update` with no `action-bindings.cpp` defining any of them, and nothing in `InputSystem` / `Application` constructs an instance. Needs: (1) the `Handle` -returning signature from this session's design doc (§9) instead of the current `void` -returning one, to avoid the two-listeners-clobber and dangling-callback-after-scene-teardown problems described there; (2) ownership inside `InputSystem` (constructed alongside `dispatcher` , ticked from `InputSystem::OnUpdate`); (3) an `Application::GetInputBindings()` accessor; (4) `Snake` (and `GameplaySceneParams` / `SnakeParams`) rewired to depend on `ActionBindings` alone rather than `IInputSystem` / `ActionSet` directly, per UE-0042/0043/0044 below.
- **UE-0042** — BUG: `Snake::Move()` 's `this->inputSystem->GetActionValue(...)` doesn't compile Two independent problems: `inputSystem` is `const Core::Input::IInputSystem&` (`snake.hpp`) — a reference, not a pointer, so `->` needs to be `.`; and `IInputSystem::GetActionValue` doesn't exist — `i-input-system.hpp` declares nothing beyond a virtual destructor, and wasn't touched this session. Fixing the `->` / `.` alone won't make this compile. Superseded by UE-0041's recommendation to route through `ActionBindings::BindAxis` instead of `IInputSystem` directly, rather than patching this call site in place.
- **UE-0043** — BUG: `Snake::Move()` looks up unregistered action names `this->inputActions.Get("MoveVertical")` / `Get("MoveHorizontal")` — but `SnakeApplication::Configure()` registers `"MoveForward"` / `"MoveRight"` , never `"MoveVertical"` / `"MoveHorizontal"` . `ActionSet::Get` asserts on an unregistered name (`action-set.cpp:26`); this aborts in a debug build even once UE-0042 is fixed. Rename one side to match the other — recommend `Get("MoveForward")` / `Get("MoveRight")` in `Snake::Move()` to match what's actually registered, unless the intent was for these to be genuinely different actions from `MoveForward` / `MoveRight` , in which case `Configure()` needs the additional `RegisterAction` calls instead.
- **UE-0044** — MAIN QUEST: `Snake` 's direction is never updated from input The old `Subscribe<InputActionPressedEvent>` / `switch` / `SetDirection` block was deleted from `Snake` 's constructor this session and nothing replaced it — `Move()` computes `vertical` / `horizontal` (once UE-0042/0043 are fixed) but never assigns `this->direction` or calls `SetDirection` with them. Movement is completely non-functional in this commit, independent of whether it compiles. Needs whatever `Move()` reads from input actually applied to direction — likely `this->SetDirection({horizontal, -vertical})` or similar, matching the axis sign convention decided this session (`W / Up` : `+1` , `S / Down` : `-1`).

- ■ **UE-0038** — MAIN QUEST: Migrate `Confirm` action off the dead `InputSystem::IsActionPressed` API Unchanged this session — `main-menu-state.cpp:31` / `game-over-state.cpp:32` still weren't touched. Once UE-0041 lands, this is exactly what `ActionBindings::Bind("Confirm", ...)` is for — no need to resurrect a pull-based query API.
- ■ **UE-0035** — MAIN QUEST: `GameContext::input` is never assigned; reconcile the `MainMenuState` workaround. Unchanged, still blocked on UE-0038/UE-0014/UE-0028 landing together.
- ■ **UE-0009** — MAIN QUEST: Implement `AppleSpawner`. Still not implemented; no longer blocked on the event bus (fixed last session).
- ■ **UE-0004** — BUG: Window-side event bus stub. Unchanged — `WindowResizedEvent` still an empty struct with no publisher.
- ■ **UE-0017** — MAIN QUEST: Wire input to event bus Needs rescoping rather than a status update: this session moved `Snake`'s *own* input consumption off the event bus entirely (the `Subscribe` call was deleted, not migrated to something equivalent), while `Confirm` / `Decline` still aren't migrated onto it either (UE-0038). The event bus itself still works correctly for digital actions (`InputDispatcher` still publishes `Pressed` / `Released` — see UE-0036); what's now unclear is whether `Snake`'s movement is meant to stay on direct polling (via `ActionBindings::BindAxis`, once UE-0041 lands) while `Confirm` / `Decline` use the event path (`ActionBindings::Bind`) — which matches this session's own design doc — or whether the intent has shifted. Recommend confirming this split explicitly next session before continuing.